12 二元搜尋樹

Part III Data Structures

本章定位

BST把全域鍵序轉成根到葉的一條查詢路徑，但普通BST不自動平衡。本章用同一棵十鍵樹對齊搜尋、後繼與雙子刪除的所有parent更新，再直接證明隨機插入的高度矩界，區分完整走訪、平均depth與height。

前置知識

遞迴、鏈結節點與隨機排列。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 建立並查詢 binary search tree。
- 實作插入與三種刪除情境。
- 分析隨機建樹的期望高度。
- 12.1 What is a binary search tree?
- 12.2 Querying a binary search tree
- 12.3 Insertion and deletion
- 12.4 Randomly built binary search trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

12.1 What is a binary search tree?

每節點含key、left、right、parent。本章主要演算法採相異key：x左子樹所有鍵<x.key，右子樹所有鍵>x.key。根parent=NIL，缺child為NIL，空樹root=NIL；必須是無環且每個非根恰一個parent的連通樹。

條件是整棵子樹的全域限制，不是只比直接child。例如根5、左子3、3的右子6，雖3<5且6>3，6仍違反「根的左子樹全部<5」。正確驗證可遞迴傳下允許區間(lo,hi)，左側改hi=x.key、右側改lo=x.key，Θ(n)檢查所有節點，不需每節點重掃子樹。

若要容重複key，可每distinct key存一個count，或以(key,identity)作全序。後者的相鄰identity可能同key，successor就不是「嚴格較大的key」。以下數例皆相異，隨機高度分析也明列此條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

中序有序性與完整走訪成本

```
inorder(x):
    if x==NIL: return
    inorder(x.left)
    output x.key
    inorder(x.right)
```

歸納：左右子樹的中序各已排序，左全部<x.key<右全部，串成左、根、右即排序。反向看，若一棵合法binary tree的所有key相異且中序嚴格遞增，左右在中序中分別位於根前後，故也滿足全域BST性質。

n個實節點各呼叫左右child一次，所以總呼叫含NIL正好2n+1。每次固定工作O(1)，給Θ(n+1)，非空樹慣寫Θ(n)，不是O(h)；每次輸出一筆也提供Ω(n)下界。遞迴stack最深h+1，鏈狀可Θ(n)。以下統一邊高：單節點h=0、空樹h=−1；非空樹路徑操作寫O(h+1)，空樹查詢與首插另為O(1)。

Heap只要求父子間大小，沒有左整棵<根<右整棵的關係，不能用同一個inorder就得到排序。

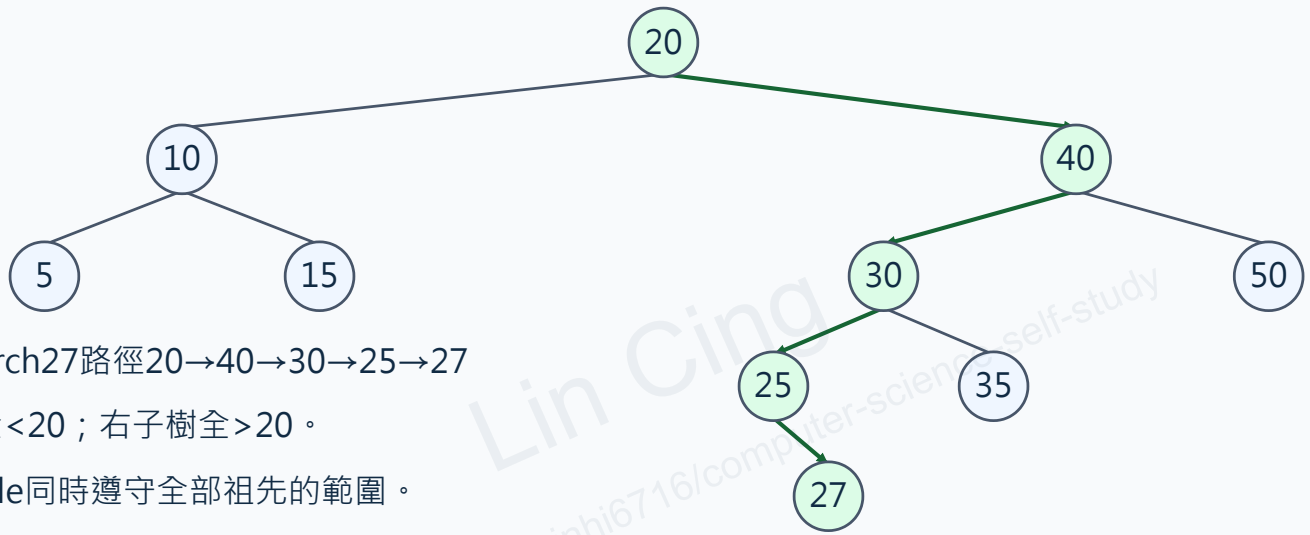
12.2 Querying a binary search tree

Search從root比較：相等回node；小於走left、大於走right，NIL回不存在。迴圈不變量是「若答案存在，必在當前x子樹」；全域BST界保證丟棄另一側不會漏答案。每步下移一層，時間O(h+1)、迭代額外空間O(1)。

Minimum在非空子樹一路走left，直到無left：若有left，左側有更小鍵；若無left，root不大於其右側全部，故可回傳。Maximum左右互換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BST查詢：路徑來自全域子樹範圍



綠：Search27路徑20→40→30→25→27

左子樹全<20；右子樹全>20。

每個Node同時遵守全部祖先的範圍。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

此例search27走20→40→30→25→27；search26到27後向left遇NIL，回不存在。Minimum=5，maximum=50。一次query沿路徑，完整inorder則為5,10,15,20,25,27,30,35,40,50，兩種成本不能混淆。

Successor的兩分支與停止理由

若x.right非空，successor=min(x.right)，因右子樹全部>x，而其最小比任何更右候選更接近。若right空，沿parent上走，持續略過「目前節點是parent.right」的邊，第一次由left上到的parent就是答案；到NIL則沒有後繼。

```
successor(x):
    if x.right!=NIL: return minimum(x.right)
    y=x.parent
    while y!=NIL and x==y.right:
        x=y; y=y.parent
    return y
```

上溯略過的parent都<x的原key，不能當successor。首次由left進入的y包含原x於其左子樹，y較大；其左子樹中原x之後沒有尚未排除的候選（原x的right空，沿途都是右側最後分支），故y最小。Predecessor把left/right對換。

例successor20走右子40的left鏈30→25，得到25。Successor27上到25時來自right，略過25；再上30時來自left，得到30。Successor50一路略過40、20，到NIL。無須重新從root按key搜尋，也沒有重新排序。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

連續Successor不能逐次粗乘Height

從minimum開始列全部n鍵，可反覆successor而總時間Θ(n)。沿完整中序過程，每條child邊至多下降一次、上升一次，加n次輸出就是O(n)；即使某一次successor走Θ(h)，昂貴路徑也不能在每個node都重新付一次。

任意起點連做k次successor（找到NIL便停止），分別數上行U與下行D。每次successor至多一條「由left child上到parent」的邊，該parent就是本次輸出的答案，共至多k條。其餘上行都從right child到parent；該parent在進其right前已輸出，而且一旦上行越過它，之後不再回到其子樹，所以每個parent最多付一次。

這些已輸出的parent，若是在本次k個答案之前就輸出，必位於起始node的祖先鏈，至多h+1個；其他至多k個。因此U≤2k+h+O(1)。每次下行depth加1、上行減1，整段的D−U等於終點depth減起點depth，至多h+O(1)，故D≤U+h+O(1)。總路徑工作D+U加k次固定開銷為O(k+h+1)，不需假定每次上行或下行都常數。

這個較細界不影響單次O(h+1)，但解釋了為何完整走訪不是O(nh)。若要自行重建每次successor都從root按key搜尋，那是另一種演算法，不能沿用上述共用邊的帳。

12.3 Insertion and deletion

Insert沿search比較路徑尋找最後NIL，保留其parent p；新節點z初始化left=right=NIL、parent=p，再依比較接p.left或p.right。空樹p=NIL則root=z。

沿路徑累積的(lo,hi)限制正是z.key可放的區間，走到NIL表示沒有同key，接葉後不影響其他鍵；新葉滿足所有祖先界，故BST性質保留。若相等則依dictionary政策更新既有record，不再接新node。

例先前樹中插26，路徑20→40→30→25→27，最後27.left=NIL，令27.left=z26、z26.parent=27。新鍵同時滿足25<26<27，不能只因26>25就任意接在25右側而覆蓋舊子樹27。

最壞O(h+1)，但以1,2,…,n插入空樹會形成右鏈，總比較0+1+⋯+(n−1)=Θ(n^2)。中序再O(n)不會把這個建樹排序變成O(n log n)。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Transplant只替換入口連結

```
transplant(T,u,v):
    p=u.parent
    if p==NIL: T.root=v
    elif u==p.left: p.left=v
    else: p.right=v
    if v!=NIL: v.parent=p
```

Transplant讓v接管u在原parent的位置，成本O(1)，但不會自動設定v.left/v.right，也不會自動刪除u的satellite data。呼叫者必須證明v子樹的key都符合原入口的全域區間；此工具不能拿任意子樹亂換。

Delete z若無left，transplant(z,z.right)，right可NIL，涵蓋leaf與只有右child；若無right，transplant(z,z.left)。原剩餘子樹全部本已在z的合法祖先區間內，所以提升後仍合法，parent同步，根刪除也由同一個工具處理。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

雙子Delete的完整指標規格

```
delete(T,z):
    if z.left==NIL: transplant(T,z,z.right)
    elif z.right==NIL: transplant(T,z,z.left)
    else:
        y=minimum(z.right)
        if y.parent!=z:
            transplant(T,y,y.right)
            y.right=z.right
            y.right.parent=y
        transplant(T,z,y)
        y.left=z.left
        y.left.parent=y
```

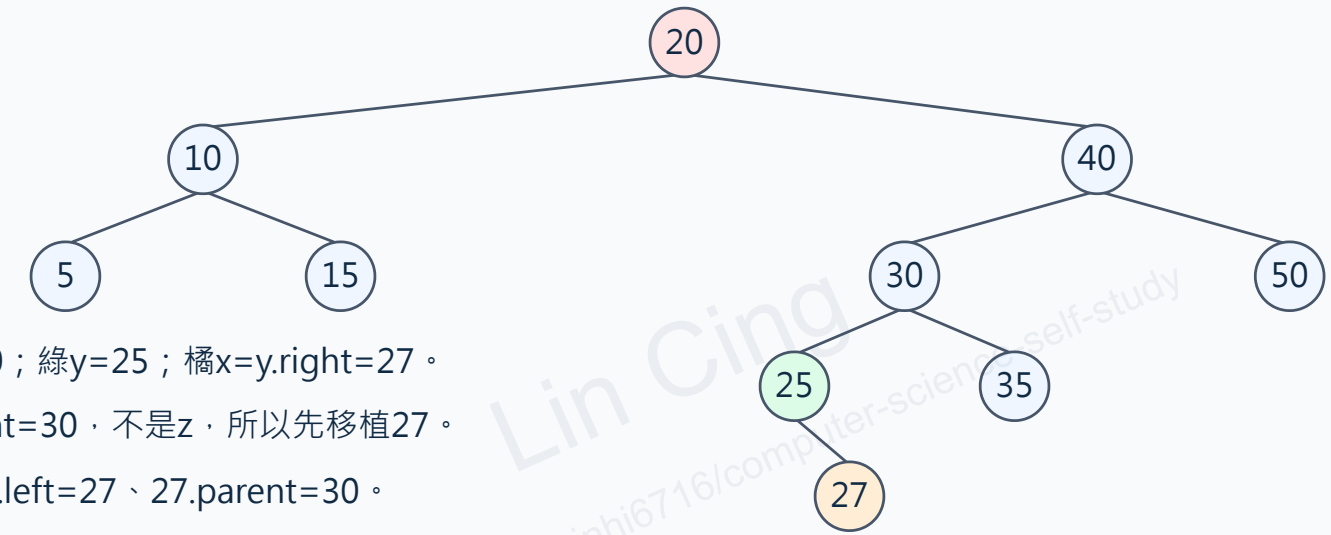
y是右子樹最小，所以無left；它可能有right，不能直接遺棄。若y不是z.right，先讓y.right接y的舊位，再讓y承接原右子樹。若y就是z.right，必須跳過這段，否則y.right=z.right會把y接到自己形成環；原y.right應原封保留。

最後y接z的原父位置（或成root），承接原left。新left所有鍵<z<y；新right是移除y後的原右子樹，全部>y。y.key本屬z的舊子樹，也符合z的全部祖先界，故全域BST性質、可達節點集合與parent方向皆正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

後繼有Right child時不能丟失它

刪除前：後繼25還有Right child27



紅z=20；綠y=25；橘x=y.right=27。

y.parent=30，不是z，所以先移植27。

先設30.left=27、27.parent=30。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

後繼有Right child時不能丟失它 （續）

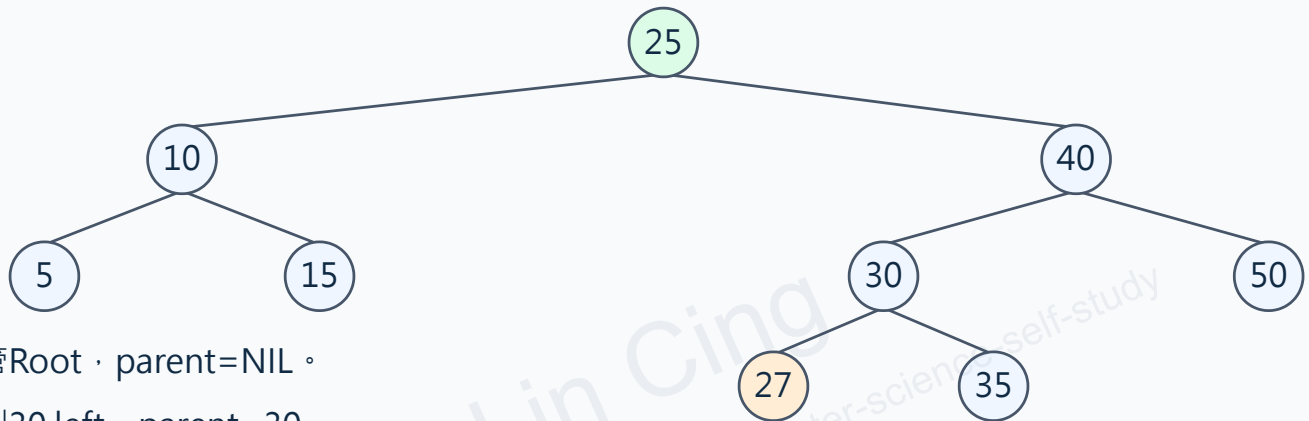
此例z=20，y=25不是直接right40，而是30.left；y.right=27。依序執行：

階段	真正修改的連結
取代y舊位	30.left=27，27.parent=30
接原right	25.right=40，40.parent=25
接原root	root=25，25.parent=NIL
接原left	25.left=10，10.parent=25

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

後繼有Right child時不能丟失它 （續）

刪除後：新Root與後繼舊位都接好



綠25接管Root，parent=NIL。

橘27接到30.left，parent=30。

只移除20，其他9個Identity都保留。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完成後只移除node20，其餘9個identity都仍可達，中序為5,10,15,25,27,30,35,40,50。這是移動node25，沒有把「複製key再刪node」的另一種handle語義混入。

刪除後的Handle與成本

傳入z必須是此樹目前有效node；delete完成後，外部不得繼續把z當作樹成員。其他node identity保持，特別是原y25只是移位置，其handle仍指同一個record。若改採複製y的key/value到z再刪y，外部handle的觀察結果會不同，需另外定義。

Delete除minimum路徑O(h+1)外只作固定次數transplant/指標更新，故O(h+1)；若從key起始先search，仍同階，但不是O(1)。搜尋、插入、刪除的好成本取決於height，普通BST沒有自動平衡保證。

若固定資料直接由已排序陣列遞迴取中點建平衡樹，可Θ(n)建好；任意未排序資料若透過比較先決定完整BST再中序輸出，就能解比較排序，故其一般最壞建樹成本至少Ω(n log n)。兩個輸入契約不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

12.4 Randomly built binary search trees

模型：n個固定相異key，由空樹開始，n!種插入排列等機率，只插入不刪除。它不同於「所有合法BST形狀等機率」。最先插入的key成root，故root秩R均勻1..n。

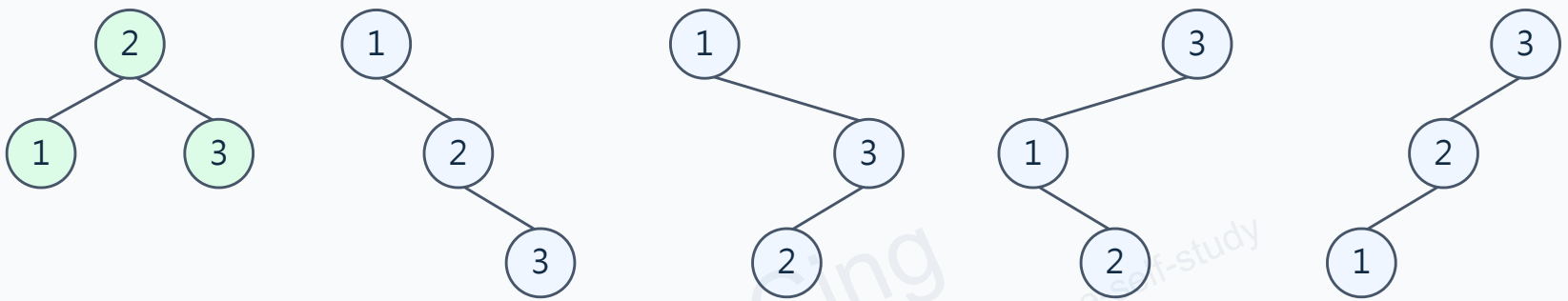
條件於R=r，左子樹含r−1個較小key，右子樹含n−r個較大key。任意指定左右內部相對排列，都有binom(n−1,r−1)種交錯方式，因此條件於root後，兩側的相對插入排列各均勻且彼此獨立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

12.4 Randomly built binary search trees （續）

均勻插入排列如何對Shape加權

六種插入排列均勻，五種Shape不是均勻。單Node邊高0。



213 / 231	123	132	312	321
2/6	1/6	1/6	1/6	1/6

綠形狀Height1，其他Height2：E[H3]=(2×1+4×2)/6=5/3。

n=3時root2的平衡形狀由[2,1,3]與[2,3,1]兩種排列產生，機率2/6；其餘四個鏈形狀各1/6。Height為1或2，$E[H_3]=(2\times1+4\times2)/6=5/3$，不是五種形狀均勻所得9/5。

先控制Exponential height

固定某一node的期望depth為O(log n)，不能直接推出全樹最大depth的期望也O(log n)。我們改直接控制height的指數。

令H_n為邊高，$H_0=-1$，Z_n=2^(H_n)，所以$Z_0=1/2$、$Z_1=1$。這和原書為方便另定$Z_0=0$的慣例不同；本章保持Z=2^H在空樹也成立，後面使用相應較寬常數。

給定root的左側大小i，height滿足H_n=1+max(H_i,H_(n−1−i))，因此z_n=2 max(Z_i,Z_(n−1−i))≤2(Z_i+Z_(n−1−i))。

設a_n=E[Z_n]。條件於root後兩子樹仍按其規模均勻建成，平均各個i得到

a_n ≤ (2/n)Σ_(i=0)^(n−1)(a_i+a_(n−1−i)) = (4/n)Σ_(i=0)^(n−1)a_i。

此上界只在max≤sum後用期望線性性，不需要誤寫E[max(X,Y)]=max(E[X],E[Y])。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

多項式Moment界與完整高度推導

取b_n=(1/2)binom(n+3,3)，$b_0=1/2=a_0$。由hockey-stick恆等式
Σ_(i=0)^(n−1)binom(i+3,3)=binom(n+3,4)，得

(4/n)Σ_(i=0)^(n−1)b_i = (2/n)binom(n+3,4) = (1/2)binom(n+3,3)=b_n。

恆等式可用Pascal式binom(i+4,4)−binom(i+3,4)=binom(i+3,3)望遠鏡求和。歸納a_i≤b_i即推出a_n≤b_n=O(n^3)，含n=1基例$a_1=1\le b_1=2$。

$\log_2$是凹函數，Jensen給$E[H_n]=E[\log_2 Z_n]\le\log_2 E[Z_n]\le\log_2 b_n=O(\log n)$。另一方面，n節點高度h最多容2^(h+1)−1筆，故每棵樹$h\ge\log_2(n+1)-1$，n增大時也給Ω(log n)。合為E[H_n]=Θ(log n)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

尾機率、適用範圍與平衡保證

對c>3，由Z_n≥0與Markov：

$Pr(H_n \geq c \log_2 n) = Pr(Z_n \geq n^c) \leq E[Z_n]/n^c \leq b_n/n^c = O(n^{(3-c)})$。

c=4給O(1/n)尾界；要失敗機率O(n^(−k))可取c≥k+3。這是隨機排列模型下的高機率界，不是普通BST對每個輸入都有固定height≤4log n。

按1..n輸入仍確定形成n−1高鏈；刪除後再插入也未必保留這個隨機建樹分布，不能把只插入的定理當成任意動態序列保證。若需要每次最壞O(log n)，應使用有明確不變量的平衡樹，例如下一章red-black tree，而不是把資料「大概隨機」當成證明。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 12。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study